

EduGenie

Google Gemini Powered Learning Assistant

PROJECT DOCUMENTATION

Project Documentation · Version 1.0 · September 2026

1. Project Overview

EduGenie is an AI-powered learning assistant designed to help students understand study material, revise key concepts, practise questions, and plan their learning. It combines a web application with AI-powered educational features using Google Gemini.

The application brings several learning tools into one place: concept explanation, text summarization, quiz generation, question answering, and learning-path support.

1.1 Problem Statement

Students may find it difficult to understand complex topics, revise lengthy notes, create practice questions, or get timely explanations. Switching between different tools can interrupt the learning process. EduGenie aims to make these activities available through a single, accessible application.

1.2 Project Objectives

- Provide AI-assisted explanations of educational concepts.
- Summarize lengthy learning material into concise notes.
- Generate quizzes to support practice and self-assessment.
- Answer students' educational questions.
- Offer a structured learning path for a selected topic.
- Provide a straightforward interface connecting users to these features.
- Organize the application into maintainable modules.

1.3 Scope

The project focuses on educational assistance through the features listed in this document. The quality and detail of generated results depend on the user's input, the configured Gemini model/service, and the implementation available in the project.

2. Main Features

2.1 Concept Explanation

Users can submit a topic or concept and request an explanation. The AI response is intended to make difficult material easier to understand. Users should verify important facts against trusted learning materials.

2.2 Text Summarization

Users provide educational text and receive a shorter summary highlighting key ideas. This can support revision and help users review long content more efficiently.

2.3 Quiz Generation

The quiz feature generates practice questions from a topic or supplied content. It is intended to support active recall and self-assessment. Generated questions and answers should be reviewed for accuracy.

2.4 Question and Answer (Q&A)

Users can ask educational questions and receive AI-generated answers. Clear, specific questions generally help the system produce more relevant responses.

2.5 Learning Path

The learning-path feature helps organize a topic into a sequence of learning steps. It can guide users through foundational ideas before moving to more advanced material.

3. System Architecture

At a high level, the application follows a web-based flow: a user interacts with the frontend, the frontend sends a request to the Python backend, the relevant module prepares the task, and the AI integration communicates with Google Gemini. The response is then returned to the interface.

Layer	Responsibility
Frontend (templates/static)	Displays pages, accepts user input, and presents results.
Backend (FastAPI / Python)	Receives requests, routes them to the appropriate feature, and returns responses.
Feature modules	Handle explanation, summary, quiz, Q&A, and learning-path tasks.
AI integration	Uses prompts/configuration to communicate with Google Gemini.

3.1 Typical Request Workflow

- The user selects a feature and enters a topic, question, or learning text.
- The frontend sends the request to the backend API.
- The backend validates the input and selects the relevant module.
- The module prepares a task-specific prompt and calls the configured AI client.
- The AI service returns generated content.
- The backend handles the response and sends it to the frontend for display.

4. Technologies Used

Technology	Purpose
Python	Backend application logic and feature modules.
FastAPI	Backend API framework (as identified in project context).
Google Gemini	AI service for generated educational responses.
HTML / CSS / JavaScript	Web interface and client-side interaction, where used.
Templates and static resources	Page rendering and frontend assets.
Git and GitHub	Version control and source-code collaboration.
pytest	Python testing, if configured in the repository.

Note: Confirm exact package versions and any additional libraries from the project's requirements.txt and source code before final submission.

5. Project File Structure

The project has been described with the following core modules and supporting folders. Names should match the actual repository; optional files may vary.

- `main.py` — Application entry point and API routes.
- `ai_client.py` — Shared client/helper for communicating with the configured AI service.
- `config.py` — Application and AI configuration.
- `prompts.py` — Prompt templates or instructions for educational tasks.
- `models.py` — Data models used by the application, where applicable.
- `explanation_module.py` — Concept explanation feature.
- `summary_module.py` — Text summarization feature.
- `quiz_module.py` — Quiz generation feature.
- `qa.py` — Question-answering feature.
- `learning_path.py` — Learning-path feature.
- `templates/` — HTML templates, if used.
- `static/` — CSS, JavaScript, images, and other static assets.
- `requirements.txt` — Python dependencies.
- `README.md` — Project overview and setup instructions.
- `documentation/` — Project documents and supporting material.

6. Installation and Setup

Use the following general setup process from the project root. Adjust commands to match the repository and the Python environment available on your computer.

6.1 Prerequisites

- Python installed on the system.
- Project source code available locally.
- A Google Gemini API key and access to the configured model/service.
- Dependencies listed in requirements.txt.

6.2 Setup Steps

- **1.** Open a terminal in the project folder.
- **2.** Create and activate a virtual environment.
- **3.** Install dependencies from requirements.txt.
- **4.** Set the required Gemini API key and configuration as environment variables or in a local `.env` file, according to the project configuration.
- **5.** Start the application using the command documented in README.md or the FastAPI entry point.

Security: Keep API keys and other secrets out of source control. Do not commit the `.env` file to a public repository.

7. User Guide

7.1 Using a Feature

- Open the EduGenie application in the configured environment.
- Choose the required tool: Explanation, Summary, Quiz, Q&A, or Learning Path.
- Enter a clear topic, question, or text as requested by the selected tool.
- Submit the request and wait for the response.
- Review the generated result. Rephrase the input or add context if the result is not useful.

7.2 Responsible Use

AI-generated output may contain errors, omissions, or unsupported details. Check important information with textbooks, course materials, or reliable sources. Avoid entering private or sensitive information.

8. Testing and Validation

Testing should confirm that the application starts, accepts valid input, returns responses for each feature, and handles invalid input or AI-service failures gracefully. Record actual results after running the tests; do not mark a test as passed unless it has been performed.

Test	Action	Expected result
TC-01	Start the application	Application starts without an unhandled error.
TC-02	Submit a question	A relevant answer is displayed.
TC-03	Request a summary	A concise summary is returned.
TC-04	Generate a quiz	Quiz questions are returned.
TC-05	Request an explanation	An explanation is returned.
TC-06	Request a learning path	A sequence of learning steps is returned.
TC-07	Submit empty/invalid input	Input is rejected or handled clearly.
TC-08	Simulate AI/network failure	A useful error is handled without crashing.

9. Limitations and Future Enhancements

9.1 Considerations

- Responses depend on the AI service, network availability, and the quality of user input.
- Generated educational content may need human review for accuracy and completeness.
- API quotas, latency, or configuration errors may affect availability.
- Actual supported file formats, user accounts, storage, and deployment features depend on the implemented code.

9.2 Future Enhancements

- Add user accounts and save learning history, if required.
- Provide difficulty levels and configurable quiz formats.
- Improve input validation, logging, and error messages.

- Add more testing and accessibility improvements.
- Deploy to a suitable hosting platform and document the live URL, once deployed.

10. Conclusion

EduGenie is designed as a Google Gemini-powered learning assistant that brings explanation, summarization, quiz generation, question answering, and learning-path support into one application. Its modular Python backend and web interface provide a foundation for further development. The project should be tested, documented against the actual source code, and configured securely before deployment.